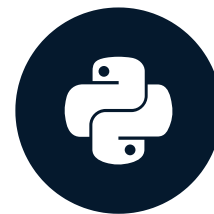


Selecting data in pandas

PYTHON FOR R USERS



Daniel Chen
Instructor

Manually create DataFrame

```
df = pd.DataFrame({  
    'A': [1, 2, 3],  
    'B': [4, 5, 6],  
    'C': [7, 8, 9]},  
    index = ['x', 'y', 'z'])  
  
print(df)
```

	A	B	C
x	1	4	7
y	2	5	8
z	3	6	9

```
df = pd.DataFrame({  
    'A': [1, 2, 3],  
    'B': [4, 5, 6],  
    'C': [7, 8, 9]},  
    index = ['x', 'y', 'z'])  
df
```

	A	B	C
x	1	4	7
y	2	5	8
z	3	6	9

```
df['A']
```

x	1
y	2
z	3

Name: A, dtype: int64

```
df.A
```

x	1
y	2
z	3

Name: A, dtype: int64

```
df[['A', 'B']]
```

	A	B
x	1	4
y	2	5
z	3	6

Subsetting rows

- Row-label (`loc`) vs row-index (`iloc`)
- Python starts counting from 0

Subsetting rows .iloc

```
df
```

	A	B	C
x	1	4	7
y	2	5	8
z	3	6	9

```
df.iloc[0, :]
```

A	1
B	4
C	7

Name: x, dtype: int64

```
df.iloc[0]
```

A	1
B	4
C	7

Name: x, dtype: int64

```
df.iloc[[0, 1], :]
```

	A	B	C
x	1	4	7
y	2	5	8

Subsetting rows .loc

df

	A	B	C
x	1	4	7
y	2	5	8
z	3	6	9

df.loc['x']

A	1
B	4
C	7

Name: x, dtype: int64

df.loc[['x', 'y']]

	A	B	C
x	1	4	7
y	2	5	8

```
df
```

```
   A  B  C  
x  1  4  7  
y  2  5  8  
z  3  6  9
```

```
df.loc['x', 'A']
```

```
1
```

```
df.loc[['x', 'y'], ['A', 'B']]
```

```
   A  B  
x  1  4  
y  2  5
```


Conditional subsetting

```
df[df.A == 3]
```

	A	B	C
z	3	6	9

```
df[(df.A == 3) | (df.B == 4)]
```

	A	B	C
x	1	4	7
z	3	6	9

Attributes

```
df.shape
```

```
(3, 2)
```

```
df.shape()
```

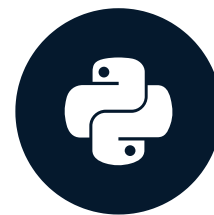
```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-17-0e566b70f572> in <module>()  
<hr />-> 1 df.shape()  
  
TypeError: 'tuple' object is not callable
```

Let's practice!

PYTHON FOR R USERS

Data types

PYTHON FOR R USERS



Daniel Chen
Instructor

R

```
df <- data.frame(  
  'A' = c(1, 2, 3),  
  'B' = c(4, 5, 6)  
)  
  
df
```

```
  A B  
1 1 4  
2 2 5  
3 3 6
```

```
class(df)
```

```
"data.frame"
```

Python

```
import pandas as pd  
df = pd.DataFrame(  
    {'A': [1, 2, 3],  
    'B': [4, 5, 6]})  
  
df
```

```
   A  B  
0  1  4  
1  2  5  
2  3  6
```

```
type(df)
```

```
pandas.core.frame.DataFrame
```

R

```
str(df)
```

```
'data.frame':   3 obs. of  2 variables:  
 $ A: num  1 2 3  
 $ B: num  4 5 6
```

Python

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 3 entries, 0 to 2  
Data columns (total 2 columns):  
A      3 non-null int64  
B      3 non-null int64  
dtypes: int64(2)  
memory usage: 128.0 bytes
```

R

```
df$A <- as.character(df$A)
str(df)
```

```
'data.frame':    3 obs. of  2 variables:
 $ A: chr  "1" "2" "3"
 $ B: num  4 5 6
```

Python

```
df['A'] = df['A'].astype(str)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
A      3 non-null object
B      3 non-null int64
dtypes: int64(1), object(1)
memory usage: 128.0+ bytes
```

String objects

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 3 entries, 0 to 2  
Data columns (total 2 columns):  
A      3 non-null object  
B      3 non-null int64  
dtypes: int64(1), object(1)  
memory usage: 128.0+ bytes
```

- When you see "object" it is a string
- Access built-in string methods with `str` accessor

String accessor

```
df = pd.DataFrame({'name': ['Daniel ', ' Eric', ' Julia '}])
df
```

```
   name
0  Daniel
1    Eric
2   Julia
```

```
df['name_strip'] = df['name'].str.strip()
df
```

```
   name name_strip
0  Daniel    Daniel
1    Eric     Eric
2   Julia    Julia
```


Category

```
df = pd.DataFrame({'name': ['Daniel', 'Eric', 'Julia'],  
                  ...:      'gender': ['Male', 'Male', 'Female']})  
df.dtypes
```

```
Out[3]:  
gender      object  
name        object  
dtype: object
```

```
df['gender_cat'] = df['gender'].astype('category')  
df.dtypes
```

```
gender      object  
name        object  
gender_cat  category  
dtype: object
```

Category accessor

```
df['gender_cat'].cat.categories
```

```
Index(['Female', 'Male'], dtype='object')
```

```
df.gender_cat.cat.codes
```

```
0    1  
1    1  
2    0  
dtype: int8
```

Datetime

```
df = pd.DataFrame({'name': ['Rosaline Franklin', 'William Gosset'],  
                  'born': ['1920-07-25', '1876-06-13']})  
  
df['born_dt'] = pd.to_datetime(df['born'])  
df
```

	born	name	born_dt
0	1920-07-25	Rosaline Franklin	1920-07-25
1	1876-06-13	William Gosset	1876-06-13

```
df.dtypes
```

```
born           object  
name           object  
born_dt    datetime64[ns]  
dtype: object
```

Datetime accessor

```
df['born_dt'].dt.day
```

```
0    25  
1    13  
Name: born_dt, dtype: int64
```

```
df['born_dt'].dt.month
```

```
0     7  
1     6  
Name: born_dt, dtype: int64
```

```
df['born_dt'].dt.year
```

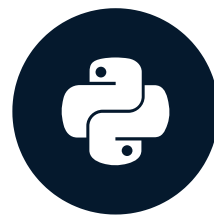
```
0    1920  
1    1876  
Name: born_dt, dtype: int64
```

Let's practice!

PYTHON FOR R USERS

More Pandas

PYTHON FOR R USERS



Daniel Chen
Instructor

Missing data

- NaN missing values from from numpy
- `np.NaN` , `np.NAN` , `np.nan` are all the same as the `NA` R value
- check missing with `pd.isnull`
 - Check non-missing with `pd.notnull`
 - `pd.isnull` is an alias for `pd.isna`

Working with missing data

```
df
```

	name	treatment_a	treatment_b
0	John Smith	NaN	2
1	Jane Doe	16.0	11
2	Mary Johnson	3.0	1

```
a_mean = df['treatment_a'].mean()  
a_mean
```

```
9.5
```


Fillna

```
df['a_fill'] = df['treatment_a'].fillna(a_mean)
df
```

	name	treatment_a	treatment_b	a_fill
0	John Smith	NaN	2	9.5
1	Jane Doe	16.0	11	16.0
2	Mary Johnson	3.0	1	3.0

More Pandas

- Applying custom functions
- Groupby operations
- Tidying data

Apply your own functions

- Built-in functions
- Custom functions
- `apply` method
- Pass in an axis

R

```
df = data.frame('a' = c(1, 2, 3),  
                'b' = c(4, 5, 6))  
apply(df, 2, mean)
```

```
a b  
2 5
```

```
apply(df, 1, mean)
```

```
2.5 3.5 4.5
```

Python

```
import pandas as pd  
df = pd.DataFrame({'A': [1, 2, 3],  
                  'B': [4, 5, 6]})  
df.apply(np.mean, axis=0)
```

```
A    2.0  
B    5.0  
dtype: float64
```

```
df.apply(np.mean, axis=1)
```

```
0    2.5  
1    3.5  
2    4.5  
dtype: float64
```

Tidy

- Reshaping and tidying our data
- Hadley Wickham, Tidy Data Paper
 - Each row is an observation
 - Each column is a variable
 - Each type of observational unit forms a table

Tidy Data Paper: <http://vita.had.co.nz/papers/tidy-data.pdf>

Tidy melt

```
df
```

```
   name treatment_a treatment_b
0  John Smith      NaN         2
1   Jane Doe    16.0        11
2 Mary Johnson    3.0         1
```

```
df_melt = pd.melt(df, id_vars='name')
df_melt
```

```
   name  variable  value
0  John Smith treatment_a    NaN
1   Jane Doe treatment_a  16.0
2 Mary Johnson treatment_a   3.0
3  John Smith treatment_b   2.0
...
```

Tidy pivot_table

```
df_melt_pivot = pd.pivot_table(df_melt,  
                                index='name',  
                                columns='variable',  
                                values='value')
```

df_melt_pivot

variable	treatment_a	treatment_b
name		
Jane Doe	16.0	11.0
John Smith	NaN	2.0
Mary Johnson	3.0	1.0

Reset index

```
df_melt_pivot.reset_index()
```

	variable	name	treatment_a	treatment_b
0		Jane Doe	16.0	11.0
1		John Smith	NaN	2.0
2		Mary Johnson	3.0	1.0

Groupby

- `groupby` : split-apply-combine
- split data into separate partitions
- apply a function on each partition
- combine the results

Performing a groupby

	name	variable	value
0	John Smith	treatment_a	NaN
1	Jane Doe	treatment_a	16.0
2	Mary Johnson	treatment_a	3.0
3	John Smith	treatment_b	2.0
4	Jane Doe	treatment_b	11.0
5	Mary Johnson	treatment_b	1.0

```
df_melt.groupby('name')['value'].mean()
```

```
name
Jane Doe      13.5
John Smith     2.0
Mary Johnson   2.0
Name: value, dtype: float64
```

Let's practice!

PYTHON FOR R USERS